POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

TERNFOLD: A Statically Typed and Expression-Oriented Programming Language

Members:

Antolin, Anicah Kim R.
Boisilio, Jewelle Melody T.
Evaristo, Yeshaya Gabriel I.
Gonzales, Daniel D.
La Madrid, Rafael Enzo T.
Liwanag, Mark Daniel L.
Montales, Mitch N.
Solomon, Nazareth Z.

INTRODUCTION

Ternfold is a proposed programming language that is centered around the principle "*Propose together. Validate in order. Publish together.*" Inspired by functional programming and the Elm Architecture, it focuses on making state changes predictable, inspectable, and safe. The language uses immutable snapshots as opposed to mutating program state so that every transition may be inspected prior to becoming the next published state. This addresses issues surrounding mutable state such as partial updates and corrupted program state.

In essence, Ternfold is a machine-based transition model wherein a state changes based on three fundamental stages: Propose, Validate, and Publish. During the proposal stage, a new candidate value is created without a change in the original snapshot. Then, these values are checked through several ordered guard conditions, and if any validation fails, the proposed state is discarded while the original state remains unchanged. All guards succeeding means that the new snapshot and its associated emissions are published as an accepted transition. Through this strategy, Ternfold removes the necessity of rolling back and enables to craft deterministic and reliable programs with predictable behavior.

BACKGROUND AND INSPIRATION

Influence / precedent	Relevant Idea	What Ternfold adopts or rejects
Lustre	Synchronous dataflow and dependency analysis	Adopts the idea that dependency structure is useful compiler work; rejects implicit clocks and real-time guarantees.

Elm architecture	Immutable model updated by messages	Adopts a pure, inspectable state-update style; extends it with explicit candidate reads and typed transition guards.
Clojure refs / transactions	All-or-nothing coordinated publication	Adopts the value-level intuition of complete publication; excludes STM concurrency, retries, and coordination machinery.
Pure functional programming	Immutable values and pure functions	Uses immutable data, closures, recursion, and explicit data-based failure.

Intended uses

- Local simulations where the next state should be inspectable and replayable.
- Rule-driven workflows where multiple validations determine whether a proposed update is accepted.
- Interactive application models, parsers with explicit domain state, and deterministic event-processing logic.
- Educational compiler work where the transition dependency graph, evaluation order, and acceptance boundary are visible.

SYNTACTIC ELEMENTS

Character sets

For the required Character Set (Sigma), it is useful to distinguish the source-language character inventory from the text-literal character domain. Ternfold source syntax is primarily ASCII; Text values may contain any Unicode scalar value subject to the string-literal rules.

Category	Ternfold 0.1 character set / rule
Letters	A-Z and a-z for identifiers. Names are case sensitive.
Digits	0-9 for decimal integer literals and for identifier characters after the first character.
Identifier marker	_ is permitted in identifiers; the standalone _ is a discard / wildcard and is not a referable name.
Whitespace	Space, tab, LF, and CRLF. Whitespace has no layout meaning.
String delimiter	" starts and ends Text literals.
String escapes	\", \\, \n, \r, \t, and \u{H...} using 1-6 hex digits, excluding surrogate code points.
Single-character punctuation	+ - / * % < > = ! & | . , ; : ( ) [ ] { } _
Multi-character tokens	:: -> => == != <= >= && || ++
Unsupported examples	No exponent operator is specified; ^ is unsupported punctuation. // is a line-comment opener, not integer division.

Identifier rules

- Pattern: ASCII [A-Za-z_][A-Za-z0-9_]*, except that standalone _ is reserved as a discard/pattern wildcard.
- Keywords cannot be used as identifiers; next is always a keyword.
- Names are case sensitive. Leading or trailing underscores do not carry visibility meaning.
- Function names and top-level types follow the same identifier character rules; pub controls export visibility, not underscore naming.

Operation Symbols

In Ternfold, there are six relational operators, three logical operators expressed with symbols, and arithmetic on Int uses checked signed 64-bit semantics.

Category	Operator(s)	Meaning / notes
Arithmetic	+ - * / %	Int addition, subtraction, multiplication, truncating division, and remainder. All are checked; overflow faults rather than wraps.

Category	Operator(s)	Meaning / notes
Text / list concatenation	++	Concatenates two Text values or two lists of the same element type.
Relational	< <= > >= == !=	Six relational/equality operators requested by the activity. Ordering applies to Int; equality is defined for equality-capable types.
Logical	! && ||	NOT, AND, OR for Bool. && and || short-circuit.

Keywords and Reserved Words

Ternfold defines 19 lexical keywords. The complete keyword set is presented below to make the language specification explicit and consistent.

Keyword	Function in Ternfold
as	Introduces the alias used after an import path.
else	Introduces the alternate branch of an if expression or the error expression of a guard.
emit	Adds a machine emission to the delayed output list after guards succeed.
enum	Declares a nominal enumerated data type.
false	Boolean literal for false.
fn	Declares a function or anonymous closure.
guard	Declares an ordered machine validation predicate with an error value.
if	Conditional expression; every if has an else block.
import	Imports a local module using an explicit relative path.
let	Declares an immutable local binding.
machine	Declares a nominal immutable state-machine type and its step semantics.
match	Pattern-matching expression; arms must be exhaustive.
next	Introduces a candidate-state writer and candidate-state projection syntax.
pub	Marks a declaration as public/exported.
record	Declares a nominal record type.
rejects	Specifies the rejection type returned by step.

Keyword	Function in Ternfold
state	Declares a machine state cell and its initializer.
true	Boolean literal for true.
yields	Specifies the emission element type of a machine.

Reserved words and compiler-known names

The type names Int, Bool, Text, Unit, List, Option, and Decision are reserved in their applicable declaration namespace. The standard-library specification also marks compiler-known prelude callables as reserved in the value namespace. The table below intentionally lists more than the required ten.

Reserved name	Role
Int	Signed 64-bit integer type.
Bool	Boolean type.
Text	Unicode scalar sequence / UTF-8 text type.
Unit	Unit value and type, written ().
List	Homogeneous finite list type.
Option	Option type with Some and None.
Decision	Machine step result type with Accept / Reject.
step	Executes a machine transition and returns Decision.
len	Returns list length.
at	Safe list lookup returning Option.
append	Adds one value to the end of a list.
map	Maps a closure left-to-right over a list.
fold	Left-folds a list.
range	Creates a half-open ascending integer range.
int_text	Converts Int to decimal Text.
bool_text	Converts Bool to Text.
parse_int	Parses a whole Int-formatted Text into Option[Int].
text_len	Counts Unicode scalar values in Text.
text_at	Returns one-scalar Text at an index, or None.
text_slice	Returns a scalar-indexed half-open Text slice, or None.

Noise Words

Ternfold for now does not define a separate class of optional “noise words” that can be inserted or removed without changing syntax. Words such as yields, rejects, else, and as read like natural-language connectors and improve readability, but they are semantically significant tokens, not optional filler.

Comments and blanks / spaces

- Single-line comment: // text until newline or EOF.
- Multi-line / block comments: not supported in Ternfold 0.1; comments are line comments only.

Blanks/Spaces

Blank lines and spaces are ignored as layout; they may improve readability but do not change parsing.

Delimiters and Brackets

Symbol	Use
,	Separates parameters, record fields, enum payload types, arguments, list items, and match arms where a comma-list applies.
;	Line / clause terminator for imports, let bindings, state declarations, next writers, guards, and emissions. Newlines do not terminate statements.
()	Function parameters/calls; () is Unit rather than a tuple.
{}	Blocks, function bodies, machine bodies, conditionals, and match arm bodies.
[]	List type application such as List[Int] and list literals such as [1, 2, 3]. Indexing with xs[i] is not a feature.
::	Qualified names such as Option::Some or module_alias::Symbol.
.	Record field projection such as p.x and candidate projection next.x.
:	Type annotation separator and named record-construction argument syntax.
->	Function result type separator.
=>	Match arm separator between pattern and arm block.

Free or Fixed Fields

Ternfold uses free-field formatting. Whitespace has no layout significance, newlines never insert semicolons, and syntactic termination is explicit through delimiters and semicolons. This permits flexible indentation while keeping machine clauses unambiguous.

Expressions and Statements

Ternfold is expression-oriented: if, match, anonymous functions, blocks, and literals all participate in expression syntax. Evaluation is strict and left-to-right except for specified conditional constructs and the machine scheduler.

Expressions

Ternfold is designed as an expression-oriented programming language. Conditional constructs such as if and match as well as anonymous functions and literal values are treated as expressions. These elements can be combined, nested, and used within the different parts of the program. This design allows the structure to be flexible and consistent.

Arithmetic expressions

```
let subtotal = price * quantity;

let total = subtotal + fee - discount;

let quotient = total / 3;

let remainder = total % 3;
```

Boolean expressions

```
let eligible = age >= 18 && verified;

let needs_review = !approved || flagged;

let safe = input != 0 && 100 / input > 2;
```

Operator precedence

Level	Forms / operators	Association
9	Calls and .field	Left
8	Unary - and !	Right
7	* / %	Left
6	+ - ++	Left

Level	Forms / operators	Association
5	< <= > >=	Non-associative
4	== !=	Non-associative
3	&&	Left, short-circuit
2	||	Left, short-circuit
1	if, match, anonymous fn, block	Primary expression forms with delimited bodies

Statements

Output statement

The **emit** expression is the output-like statement where emitted values are evaluated after the guard conditions are met. It is returned in the same order to what appears in the machine.

```
machine Counter(delta: Int) yields Int rejects Text {
    state count: Int = 0;
    next count = count + delta;
    guard next.count >= 0 else "negative count";
    emit next.count;
}
```

Input statement

Ternfold does not use a separate input or read statement. A machine receives the input through the header parameter, while normal functions are received through their own parameters.

```
machine Counter(delta: Int) yields Int rejects Text {
    state count: Int = 0;
```

```
        next count = count + delta;
        guard next.count >= 0 else "negative count";
        emit next.count;
    }
```

Assignment statement

Ternfold does not support a mutable assignment. The nearest equivalent to this is an immutable let binding and a machine update which uses a next writer.

```
    let result = x * 2;
    result
```

Conditional statement

Every if statement in this programming language requires an else block. When checking another condition in an else-if chain, it is written as an if expression nested inside the else block and only the branch that satisfies the condition will execute.

```
    let label = if count >= 10 {
        "full"
    } else {
        "open"
    };
```

Input, process, output in one example

```
    pub fn main() -> List[Text] {
        match step(Counter(), 4) {
            Decision::Accept(counter, values) => { values },
            Decision::Reject(counter, reason) => { [reason] }
        }
```

```
}
```

TERNFOLD - PROPOSAL AMENDMENT | REVISION 2

Course-aligned syntactic elements, finite automata, and execution examples

Scope of this amendment

The original 11-page group proposal has been retained page-for-page without modification. The following pages add the requested finite-automata diagrams and resolve the rubric items that Ternfold 0.1 explicitly did not support. Where these pages differ from the original pages 4, 7, 9 or 10, the revised 0.2 specification on these amendment pages takes precedence.

Design identity remains Ternfold

Ternfold still follows Propose together. Validate in order. Publish together. Its immutable machines, ordered guards, compiler-scheduled candidate dependencies, and delayed emissions are unchanged. New coursework syntax is purposefully constrained to preserve that identity.

Course requirement	0.1 proposal	Revision 2 normative addition
Exponent and distinct integer division	Neither ^ nor // as division	Checked ^ power; div floor quotient. // remains a comment.
Noise words	Not supported	Optional readability phrases from stdin and to stdout.
Block comments	Not supported	Nonnesting /* ... */ alongside //.
At least 4 lexical constants	Int / Bool / Text (Unit is parsed)	New Char literal gives 4 independent lexical categories.
Input / output statements	Function input / emit as analogy	Explicit input/output clauses in public main; input captured, output staged.
Single / nested loops	Recursion / map / fold only	Pure for-in list comprehensions, freely nested.
Finite automata + symbol table	No diagrams/export	DFA diagrams and a real text export required by roadmap.

All proposed additions are compiler targets, not claims that the revised language has already been implemented. The associated restart repository specifies their semantics and acceptance tests.

I. CHARACTER SET (SIGMA) AND OPERATIONS

Ternfold source syntax is primarily ASCII. UTF-8 is required; quoted Text and Char values can contain Unicode scalar characters. Names are case sensitive.

Character category	Exact source inventory / rules
Uppercase letters	ABCDEFGHIJKLMNOPQRSTUVWXYZ
Lowercase letters	abcdefghijklmnopqrstuvwxyz
Digits	0123456789
Identifier marker	_ : allowed inside names; standalone _ is discard/wildcard
Arithmetic/operators	+ - * / % ^ < > = ! & |
Brackets/delimiters	( ) [ ] { } , ; : .
Quote/escape marks	" (Text), single quote (Char), backslash for escapes
Whitespace	Space, tab, LF and CRLF; free-field; newlines never end clauses
Longest-match tokens	::, ->, =>, ==, !=, <=, >=, &&, ||, ++, // and /* ... */
Other source glyphs	Rejected outside strings/comments; diagnostic + INVALID report row

Arithmetic operators (all Int; checked)

Spelling	Meaning and example	Semantics
+ - *	Add, subtract, multiply	Checked signed 64-bit.
/	-7 / 3 = -2	Integer quotient truncated toward 0.
div	-7 div 3 = -3	Integer quotient floored toward negative infinity.
%	-7 % 3 = -1	Remainder consistent with truncating /.
^	2 ^ 3 ^ 2 = 512	Right-associative power; nonnegative exponent only.

Other operation symbols

Relational: < <= > >= == != (six). Logical: && (AND), || (OR), ! (NOT), with short-circuit behavior. `++` concatenates Text or equal-element-type lists.

Identifiers follow ASCII `[A-Za-z_][A-Za-z0-9_]*` except standalone _. Both English names such as `student_count` and algebraic function names such as `square` and `polynomial` use this rule.

II. KEYWORDS, RESERVED NAMES, NOISE WORDS

The original 19 keywords remain valid. Revision 2 introduces div, for, in, input, output, from, to, stdin, stdout; it does not repurpose previously valid identifiers without an explicit grammar change. The complete revised keyword set contains 28 entries.

Keyword	Function	Keyword	Function
as	Import alias	div	Floor integer quotient
else	Alternate branch / guard error	emit	Delayed machine output
enum	Nominal variants	false	False Bool constant
fn	Function or closure	for	List-producing iteration
from	Optional noise connector	guard	Ordered validation
if	Conditional expression	import	Relative module import
in	Iterable separator	input	Main-only input clause
let	Immutable assignment binding	machine	State transition declaration
match	Exhaustive case expression	next	Proposed cell writer/read
output	Main-only output clause	pub	Export visibility
record	Nominal record type	rejects	Machine error type
state	Machine state cell	stdin	Entrypoint source marker
stdout	Entrypoint sink marker	to	Optional noise connector
true	True Bool constant	yields	Machine emission type

Reserved names (at least 10)

Reserved types: Int, Bool, Text, Char, Unit, List, Option, Decision. Reserved prelude names: step, len, at, append, map, fold, range, int_text, bool_text, parse_int, text_len, text_at, text_slice. Redeclaration in their corresponding namespace is prohibited; these are not all individual lexer keyword tokens.

Genuine optional noise words

`input user: Text from stdin;` and `input user: Text;` are equivalent. Likewise `output user to stdout;` and `output user;`. The connector words from/to improve readability but have no extra runtime effect; stdin/stdout are fixed designators. Their use outside the defined phrases is invalid.

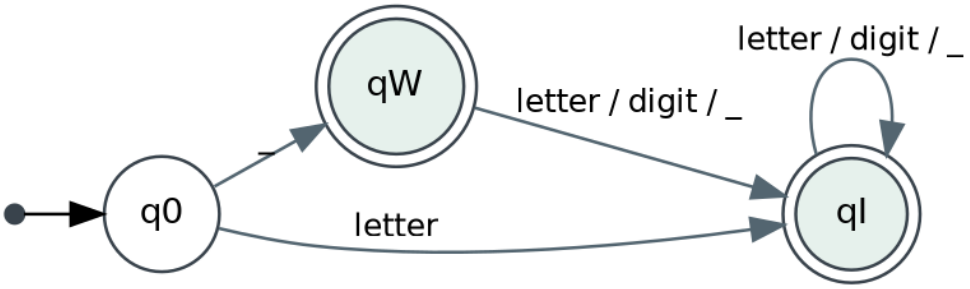
Blanks, comments, delimiters and field format

Single line: `// comment`. Block: `/* comment */` (nonnesting, unterminated comment is an error). Spaces, tabs and line breaks are free-field trivia. Comma separates list/argument items; semicolon terminates bindings and clauses; opening and closing brackets are ( ), [ ], and { }.

III. FINITE AUTOMATA - IDENTIFIERS AND INTEGERS

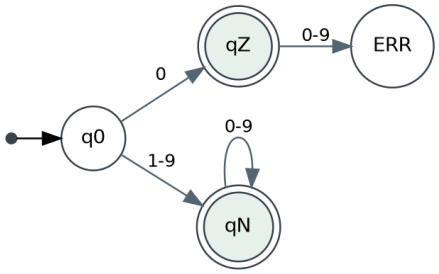
The required model is a deterministic finite automaton (DFA) for lexical recognition, not Ternfold's separate machine-transition model. Accepting states are drawn with double outlines. The scanner uses longest-match and preserves source byte spans.

A. Identifier / wildcard DFA



Transition rules: q0 --letter--> qI; q0 --underscore--> qW; qW --letter/digit/underscore--> qI; qI loops over letter/digit/underscore. qI accepts identifier; a solitary qW lexeme accepts wildcard underscore. After accepting, full-lexeme lookup classifies keywords (case-sensitive).

B. Decimal integer DFA



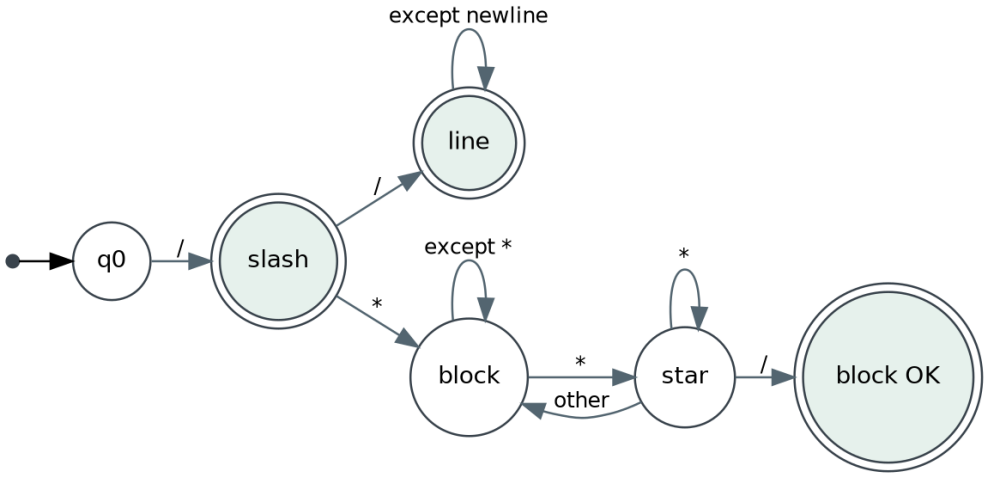
Transition rules: q0 --0--> qZ (valid zero); q0 --1..9--> qN (valid nonzero integer); qN loops on 0..9; a digit after qZ enters error state (leading zero E102). A leading minus is a separate operator token.

Input	Automaton result	Token/report category
student2	qI accepting	IDENTIFIER
_	qW accepting	WILDCARD
machine	qI then exact keyword lookup	KEYWORD
Int	qI then reserved-name overlay	RESERVED_NAME
0	qZ accepting	INT_LITERAL
024	error E102	INVALID

Formal interpretation: a lexer DFA is a 5-tuple (Q, Sigma, delta, q0, F). Q is the finite collection of labeled circles; Sigma is the relevant set of character classes; delta is the transition function; q0 is the start; F contains the double-outlined accepting states. Any excluded transition rejects the attempted lexeme.

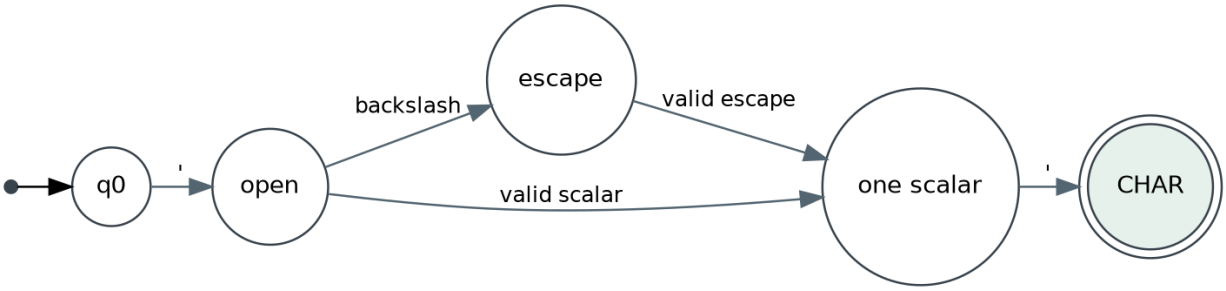
IV. FINITE AUTOMATA - COMMENTS AND CHAR

C. Division, line-comment, and block-comment DFA



At /, the scanner provisionally accepts arithmetic slash. // begins a comment ending at newline or EOF. /* begins a nonnested comment that closes only at the first */. End-of-file while in the block body or star state is invalid E104. Integer division is written div, avoiding conflict with // comments.

D. Character-literal DFA

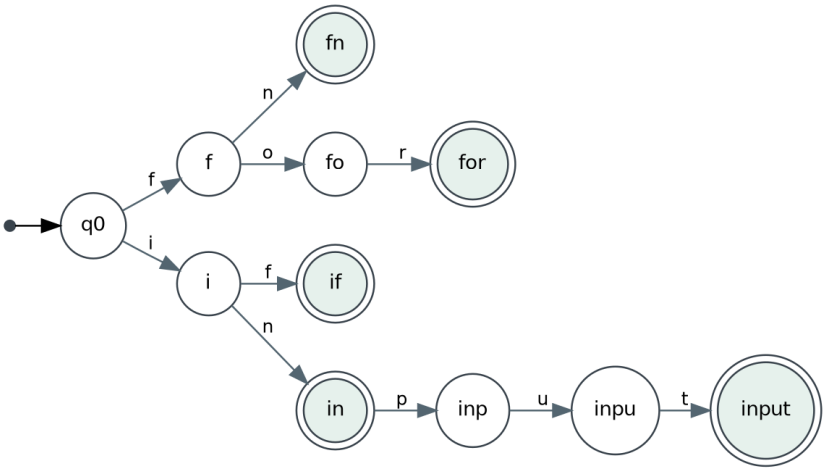


A single quote starts the literal. A valid Unicode scalar or approved escape supplies exactly one value; the closing single quote accepts the Char token. Empty or multi-scalar values, malformed escapes, raw controls, surrogates, or an absent closing quote yield E105. Text uses a similar repeated-scalar/escape recognizer terminated by double quote.

Input	Expected recognition
'T'	CHAR_LITERAL
'Ω'	CHAR_LITERAL (one Unicode scalar)
'\n'	CHAR_LITERAL (escaped scalar)
'' / 'ab'	INVALID E105
/* test */	COMMENT_BLOCK
// test	COMMENT_LINE
/* unfinished	INVALID E104

V. FINITE AUTOMATA - KEYWORDS AND OPERATORS

E. Keyword-trie sub-automaton (illustrative prefix family)



This exact-match trie recognizes the keyword family fn / for / if / in / input after reading complete word boundaries. Other keyword spellings use the same deterministic prefix method; identifier matching comes first, so `format` is one IDENTIFIER rather than keyword `for` followed by text. Double circles are accepting spellings; a longer identifier continues in the identifier DFA.

F. Reserved names versus keywords

Keyword spelling is recognized lexically after an identifier candidate. Reserved prelude/type names such as Int or range are valid identifier-form lexemes and are rejected only when redeclared in prohibited namespaces. The educational Symbol Table text export labels those as RESERVED_NAME by a documented overlay; the production parser still receives correct token kinds.

G. Prefix-operator recognition

Prefix glyph	Single token	Longer token(s)
:	:	::
-	-	->
=	=	==, =>
!	!	!=
< / >	< / >	<= / >=
& / |	invalid solitary	&& / ||
+	+	++
/	/	// line-comment, /* block-comment
^	^	none

H. Finite-automata verification

The implementation must test every transition family, valid/invalid/EOF boundaries, Unicode byte spans, comment delimiters, keyword/identifier ambiguity, and maximal-munch precedence. This finite-automata appendix is not a claim that the compiler already performs all recognition correctly.

VI. EXPRESSIONS AND STATEMENTS - COURSE EXAMPLES

Precedence (higher before lower)

Order	Operations	Associativity
1	Calls; field selection	Left
2	^ power	Right
3	Unary - and !	Right
4	* / div %	Left
5	+ - ++	Left
6	< <= > >= and == != (separate levels)	Non-associative
7	&& then ||	Left, short circuit

Arithmetic + Bool examples

```
let power = 3 ^ 2; let quotient = -7 div 3; let keep = count > 0 && approved;
```

Single and nested loops (pure; no mutable index)

```
let doubles = for n in range(0, 4) { n * 2 };
```

```
let grid = for row in range(0, 2) {
    for col in range(0, 3) { row * 10 + col }
};
```

Entrypoint input, immutable assignment, output, and conditional

```
pub fn main() -> List[Text] {
    input name: Text from stdin;
    let grade: Int = 3 ^ 2;
    let result = if grade >= 9 { "pass" } else { "retry" };
    output "Hello " ++ name to stdout;
    [result]
}
```

For the input line Ada, the full evaluation publishes Hello Ada followed by pass. Input is taken from a captured immutable snapshot and output is buffered until main succeeds. Machine `next cell = expression;` remains a candidate assignment, not mutable reassignment. An if-else-if chain nests another if inside else.

Four independently lexed literal categories

Int 17; Bool `true`; Text `"hi"`; Char `'q'`. The fifth Unit value `()` is recognized at the parser level from two bracket tokens.

VII. ACCEPTANCE CROSSWALK AND IMPLEMENTATION NOTES

The reference Group-2 proposal provided a useful example of placing word-recognition state diagrams beside syntax tables. These are newly designed Ternfold automata, not imported diagrams from another language.

Course group	Proposal evidence	Compiler acceptance owner
Identity/inspiration	Original pages 1-3	M00/C00 documentation
Sigma/naming/keywords/reserved	Original pages 3-6 + amendment II, III	M03 lexer, M05 resolver
Operators, precedence, constants	Amendment II, IV, VI	M03, M04, M06, M09, M11
Finite automata	Amendment IV-VI diagrams	M03 transition tests
Noise words / comments	Amendment III, V	M03/M04 with negative tests
Input/output/assignment	Amendment VII	M04/M06/M08/M09/M11
Conditional / single/nested loops	Original pp. 8-10 + amendment VII	M04/M06/M09/M11
Wrong input file extension	Symbol-table CLI requirement	M03/CLI
Invalid token and text symbol table	Report categories + E10x faults	M03 lexer/report tests
Native interpreter/backend equivalence	Pure, replayable driver boundary	M09/M11/M14

Symbol Table.txt output contract

The lexer will generate a UTF-8 text file listing token/lexeme, class, line/column and diagnostic, without replacing real compiler tokens. Labels include IDENTIFIER, KEYWORD, RESERVED_NAME, NOISE_WORD, INT_LITERAL, BOOL_LITERAL, TEXT_LITERAL, CHAR_LITERAL, COMMENT_LINE, COMMENT_BLOCK, OPERATOR, DELIMITER and INVALID. Reject source files with an extension other than .tfd. Example syntax: `ternfold tokens file.tfd --symbol-table "Symbol Table.txt"`.

Final accuracy statement

These pages finalize the proposed Ternfold 0.2 language contract. Actual compiler correctness requires future implementation and tests; this addendum and the restart ZIP are not evidence that M03-M14 have passed. The language identity, ordered machine guards, candidate DAG, static types and C17/Python architectural commitments remain intact.

TERNFOLD - CURRENT COURSE CLARIFICATION

28 October 2026 | Applies after the preserved original and revision-2 pages

Precedence: These pages supersede conflicting earlier numerical, word-recognition and classroom-report statements. The original group proposal remains preserved. Machine snapshots, ordered guards and delayed emissions are unchanged.

1. Required operations and numerical vocabulary

The rubric counts are operations, not points: seven arithmetic operations and nine Boolean/relational operations. Every spelling is independently recognized and tested.

Group	Exact spellings
Arithmetic (7)	+ - / * % div ^
Boolean/relational (9)	&& || ! > < == != >= <=

/ is real division: Int/Float operand pairs produce finite IEEE binary64 Float, with explicit nearest/ties-even conversion for Int operands. div is Int-only floor division; % retains the dividend-sign remainder. Other numeric mixing is strict except Float raised to an Int exponent. No NaN or infinity values exist. Numerical execution is Stage 2, not performed by the Stage 1 lexer.

Literal categories: Int, Float, Bool, Text and Char are five genuine lexical families. Unit () remains two parenthesis tokens. Float literals include 1.5, 1e3 and 1.25e-2; minus is a separate token. Malformed numeric runs such as 01, 1e+ and 1.2.3 are INVALID. The lexer preserves magnitudes; Int64/finite Float checks belong to semantic analysis.

Reserved-name additions: Float, int_float, float_int and float_text join all 21 prior proposal names; none is removed. The full current inventory has 25 reserved names and the same 28 lexical keywords.

2. Strict character-state recognition

A handwritten character scanner and finite prefix DFA recognize identifiers, keywords and reserved names. Each consumed character advances a transition; terminal metadata at the identifier boundary supplies its category. No completed-word dictionary lookup or spelling-set match is used. Keyword continuations such as letter, format and input2 are one identifier. Regex tokenization, premade analyzers/generators, eval and exec are absent.

This replaces page 15 wording about full-lexeme lookup. The prefix method on page 17 now applies to every keyword and reserved name. Decimal recognition also uses explicit states; comments and ambiguous operator prefixes obey maximal munch.

STAGE 1 LEXICAL ANALYZER SUBMISSION

3. Classroom Symbol Table.txt

The actual UTF-8 text file is named exactly Symbol Table.txt and contains exactly two tab-separated columns: TOKEN TYPE and LEXEME. Rows preserve source order, including line/block comments and invalid runs. Whitespace and EOF are omitted; Unit does not duplicate parentheses.

The lexeme column uses quoted reversible escaping for tabs, line endings, quotes, backslashes and Unicode. A multiline comment stays one table row; decoding the escaped column recovers the exact original lexeme. Detailed byte/line spans remain in the internal token API and do not add classroom columns. Invalid encoding prevents a source snapshot and yields a header-only table plus an original-byte diagnostic.

4. Complete mandatory lexical checklist

1. Only .tfd inputs, exact extension case.

2. ASCII identifiers, case sensitivity, keyword/identifier boundaries.

3. All 28 keywords independently tested.

4. All 25 reserved names, including every printed revision-2 name.

5. Five literal families and invalid literal recovery.

6. Contextual from/to noise words in matching input/output phrases.

7. Single-line // and nonnesting block /* */ comments.

8. All seven arithmetic and nine Boolean/relational operations.

9. Independent delimiters and brackets with maximal munch.

10. INVALID rows, bounded diagnostics and scanner progress.

11. Actual two-column Symbol Table.txt generated and compared to expected bytes.

5. Delivery and verification scope

The standalone source/launchers/examples/report/tests run offline with the Python standard library only. No GCC, compiler analysis/Core/runtime, network, AI service or development package is needed. Python 3.12-3.14 syntax is supported; executed verification is Windows Python 3.14.4. POSIX execution is not claimed. The ZIP is tested after extraction outside the compiler checkout in a fresh pip-free environment. Stage 2 continues the full compiler ecosystem separately.

For the presentation: prepare the physical INDEX CARD and print this latest proposal, including these clarification pages. The lexical analyzer does not claim parsing, type checking, execution or native parity.